

摘要：

DeepSeek开源DSpark仅一周，工程师Abdur Rahim便推出首个苹果芯片原生版本mlx-dspark。在M4 Pro上，该版本通过投机解码与MLX框架重构，成功将Gemma-4和Qwen3模型的生成速度分别提升1.6倍和1.4倍。

DSpark刚开源一周，就被搬进了苹果电脑。

移植版本叫mlx-dspark，跑的是Gemma-4 12B和Qwen3-4B这两个模型。

装上之后，这两个模型在Mac上的生成速度分别提了1.6倍和1.4倍。



Abdur Rahim

@_ARahim_

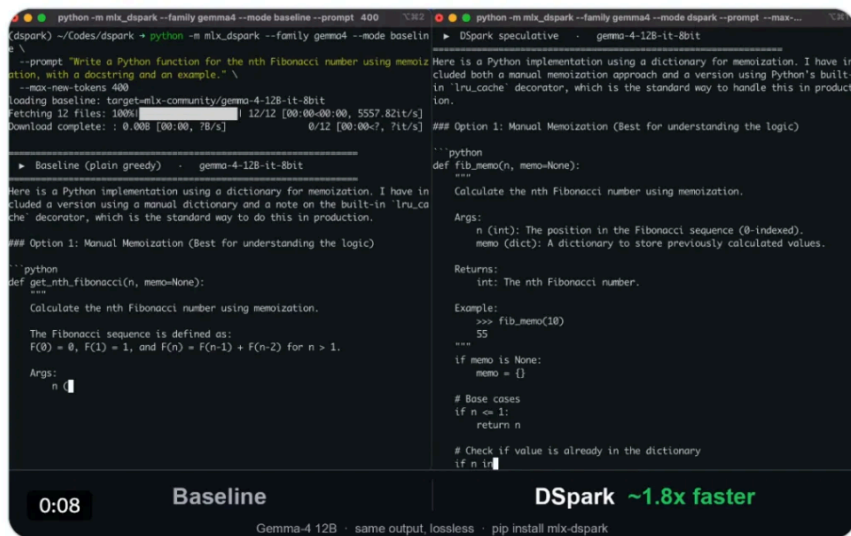
显示翻译

DeepSeek's DSpark speculative decoding now runs on Apple Silicon 🎉

I ported it to MLX, so the published drafter checkpoints run natively on a Mac.

Same output as the base model, just faster.

@googlegemma @deepseek_ai



上午2:17 · 2026年7月1日 · 2.2万查看

公众号 · 量子位

更难的是，它做到了大多数移植版本做不到的一件事——输出和原模型逐字节相同，一个字都不差。

也就是说，速度换来了，质量一点没丢。

动手的人是Abdur Rahim，业余时间捣鼓开源项目的一个工程师，DSpark开源以来的第一个Mac原生版本，都是他一个人做出来的。

苹果电脑跑大模型，提速60%

针对DeepSeek在6月27日开源的DSpark，官方给出的数字是服务端场景下能提速60%到85%。

不过这套技术当时只有数据中心GPU上的实现，没有适配苹果芯片的版本。

mlx-dspark是这套技术的第一个苹果芯片原生版本。

mlx-dspark

DeepSeek's DSpark and z-lab's DFlash speculative decoding — running natively on Apple Silicon via [MLX](#).
Lossless drafters that make Gemma-4 12B and Qwen3-4B faster on a Mac — same output, with a built-in DSpark-vs-DFlash head-to-head (DSpark ~1.6x / ~1.4x; DFlash up to ~2.1x on code/math).

 [v0.0.3](#)  [python 3](#)  [platform](#) [Apple Silicon](#)  [license MIT](#)

 公众号 · 量子位

DSpark的思路是配一个更小的模型给目标模型打下手，小模型先一口气蹦出几个候选词，目标模型再一次性核对，对的收下，错的打回去重猜。

这一步的成本，在数据中心和苹果电脑上不一样。

在数据中心的GPU上，核对一批候选词更像包车，坐几个人都是一口价，解码本来就是内存瓶颈，多核对几个词几乎不多花时间。

苹果芯片更像打表的出租车，核对的候选词越多，表跳得越多。

Rahim实测过，Gemma-4 12B每多核对一个token，要多花约14毫秒。他把这套账算成了一个成本模型，得出的结论是，苹果芯片上的速度天花板在2.2倍左右。

总之，Rahim把这个打下手的小模型从HuggingFace的checkpoint里搬了过来，分别配给Gemma-4 12B和Qwen3-4B这两个目标模型使用。

他还把核对流程在MLX框架里重新搭了一遍，权重量化成4-bit。

How it works

- A **parallel backbone** (5 Gemma-4 layers) consumes the target model's hidden states (tapped at layers [5, 17, 29, 41, 46], EAGLE3-style) and proposes a 7-token block at once.
- A **rank-256 Markov head** adds a previous-token correction, sampled sequentially — the only sequential cost, which kills "suffix decay" cheaply.
- A **confidence head** scores each draft position (optional adaptive block length).
- The target **verifies** every token, so output is **greedy-correct by construction** (identical to plain greedy decoding, up to floating-point tie-breaking on near-ties).

结果，在M4 Pro上，对比苹果官方的MLX工具，Gemma-4 12B的生成速度从18.4tok/s涨到约30tok/s，是原来的约1.6倍；Qwen3-4B从52.9tok/s涨到约73tok/s，是原来的约1.4倍。

另外，在mlx-dspark里，Rahim还做了一件大多数移植工作没做的事。

移植版本，也能高精度还原

多数把大模型搬到本地的版本，只支持贪婪解码，也就是每一步都挑概率最高的那个词。

Rahim在mlx-dspark里，把DSpark论文里原本描述的温度采样方法也实现了出来，草稿模型给出候选词，接受概率是 $\min(1, p/q)$ ，没通过的部分从残差重新采样。

他自己核对过，这套流程跑出来的输出，严格等于目标模型在同样温度下会给出的那个精确分布，不是打了折扣的近似版本。



多数投机解码只做贪婪版本，是因为验证贪婪模式的正确性很简单，逐字比对就行。

Rahim多做的这一步，是自己把采样模式下跑出来的输出分布核对了一遍，确认没有走样。

负责核对的目标模型该配哪个精度，是他自己试出来的一个坑。

如果小模型配的是没经过指令微调的基础版目标模型，蹦出的候选词只有47%能通过核对；换成对应的指令微调版本，这个比例涨到82%。

他还测过把目标模型换成bf16精度，核对成本涨得比通过率涨得多，反而更慢，所以目标模型默认留在8-bit上最划算。

负责打前站蹦候选词的小模型，用的是另一套精度。

草稿模型本身被他做了压缩，4-bit量化之后只有1.8GB，装进内存毫无压力，跑起来还是无损。

结果就是，DSpark不仅实现了加速，也确实把论文里提到的16%到18%接受率提升，在设备端复现了出来。

DFlash也接了进来，代码任务更快

推文发出后，评论区来了一条留言，DFlash论文的作者之一Jian Chen问，能不能试试他们团队的模型。

DFlash是z-lab今年5月发的论文里提出的另一种投机解码方案，作者团队带头人Zhijian Liu，UCSD助理教授，同时是NVIDIA的研究科学家。



DFlash的思路和DSpark不太一样，它用一次并行的「块扩散」去噪一整块16个token，而不是像DSpark那样一步步带着依赖关系去猜。

Rahim迅速动手。

他用Jian自己写的移植脚本，把z-lab发布的gemma4-12B-it-DFlash接到mlx-vlm的Gemma-4目标模型上，在同一台Mac上，跟自己刚测完的DSpark又跑了一轮头对头对比。

代码和数学任务上，DFlash整块解码的接受长度能到5.95到6.20，速度约36tok/s，达到约2.1倍，跑赢了DSpark。

method	chat	code	math
DSpark (cap 2)	2.45 / 28.5	2.78 / 32.8	2.86 / 32.4
z-lab DFlash (cap 2)	2.15 / 24.2	2.76 / 31.3	2.71 / 29.6
z-lab DFlash (full 16)	2.68 / 16.9	5.95 / 36.6	6.20 / 36.3

但是，DFlash一次要蹦出一整块16个token，而目标模型未必全部认可，实际能通过核对的只是其中一部分，业内管这个叫“接受长度”，不是每次都能把16个全填满。

所以在开放聊天这种内容不好预测的场景里，接受长度上不去，块填不满，DFlash的优势发挥不出来。

DSpark的Markov头正是为了对付同一个毛病存在的，并行蹦出一整块词，越往后的位置是各自独立算出来的，容易互相不搭调，Markov头给这些位置之间加了一层依赖关系，专门纠正这个问题。

结果就是，在聊天场景里，DSpark反而比DFlash更快。

**Abdur Rahim** @_ARahim_ · 7月1日

Thanks Jian, I ported your gemma4-12B-it-DFlash into MLX (ran your model_mlx.py against an mlx-vlm gemma-4-12B-it-8bit target, M4 Pro, greedy/lossless) and benchmarked it head-to-head vs DeepSeek's DSpark on the same target/Mac, 4 prompts/domain, warm, accepted-length / tok·s:

Code/math, full block-16: DFlash hits 5.95 / 6.20 accepted at ~36 tok/s (~2.1×), beating DSpark on both acceptance (its block-7 ceiling ~2.8) and throughput. Your block-diffusion denoise is the clear winner on structured content.

Open chat: the block doesn't fill (accept ~2.2-2.7), so DSpark's markov tail-correction edges ahead, right where the paper's suffix-decay framing predicts.

One Apple-Silicon caveat: multi-token verify drops off MLX's fast quantized-GEMV path, so per-token verify cost grows, and the long block only converts to wall-clock when acceptance is high (code/math), your b16 strength wants the cheap-verify batched-serving regime more than a single Mac.

It was fun to reproduce. One thing to ask for the single-user case: is there a recommended way to run b16 at a shorter effective block when verify isn't free, or do you treat that as out-of-scope vs the serving setting DFlash targets?

 2 326

而后更新的mlx-dspark v0.0.3，正式把z-lab原版DFlash接入了包里，还加了一个参数，可以手动把DFlash的有效块长度调短，聊天场景用短块，代码和数学场景仍然用满16的整块。

这之后，同一台Mac、同一个包，就能同时完成聊天和代码、数学类的任务，不用再在DSpark和DFlash两个项目之间来回搬了。



Rahim在推文里说，同样的方法，用在更大的Qwen3-8B和14B草稿模型上应该也能跑通。

来源：[量子位](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 158  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论